EE4-52
EE9AO15

IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING
EXAMINATIONS 2012

MSc and EEE PART IV and ISE PART III: MEng and ACGI

Corrected Copy

P.3. 11.00

EMBEDDED SYSTEMS

Friday, 18 May 10:00 am

Time allowed: 1:30 hours

There are TWO questions on this paper.

Answer BOTH questions.

The questions carry equal marks.

Any special instructions for invigilators and information for candidates are on page 1.

Examiners responsible First Marker(s) : T.J.W. Clarke
Second Marker(s) : Y.K. Demiris

© Imperial College London

Special Information for Invigilators: none.

Information for Candidates

The Rate Monotonic Analysis condition for a set of N jobs where the ith job has CPU utilisation U_i is:

$$\sum_{1}^{N} U_i < N\left(2^{1/N} - 1\right)$$

© Imperial College London

The Questions

Both questions are compulsory and carry the same number of marks

1.

a) Two engineers are arguing about how to design and test safety-critical software running under a RTOS. The design specification includes hard deadlines.

A: "We've tested the equipment for 10,000 hours under typical use conditions with only 5 logged incidents of software failure, all corrected by the safety task we added after incidents were first reported from field trials. We have separately tested the safety task effectiveness in a simulation and it fails to correct less than 1 in 10,000 software failure incidents. Therefore the overall rate of uncorrected incidents is now less than 1 in 20,000,000 hours."

B: "Your safety analysis is unsound, and design strategy inappropriate for the application."

Discuss fully the merits of B's view, giving specific reasons to justify your statements.

[10]

b) Many RTOSs define so-called "mutex" semaphores in their application programming interface as distinct from simple binary semaphores. State TWO distinct features that might be useful in a mutex semaphore, not always found in simple binary semaphores. Describe concisely:

(i) How each feature works?

[2]

(ii) Why is each feature useful?

[2]

(ii) One aspect of each feature's implementation which is more complex and less efficient than a simple binary semaphore.

[2]

c) You are asked to determine the speed of a new faster CPU for an embedded system controlling a personal video recorder. The new CPU will be identical to the old in all aspects except speed. The existing system has three tasks running under a RTOS without significant blocking and with measured CPU utilisation between 5% and 80% depending on system load. System developers tell you that future functionality upgrades are likely to increase the overall CPU workload by no more than 100%. State, making appropriate and explicit assumptions as necessary, what is the minimum safe speed increase for the new CPU.

[4]

2.

a) Figure 2.1 displays the CPU time for the computation associated with each job *A*, *B*, *C*, *D* in a real-time system. Each job is implemented as a separate task under a priority--based RTOS. Using rate monotonic analysis, or otherwise, determine the minimum safe speed for the CPU in cycles per second, assuming no inter-task blocking and perfect task switching. State inequalities on the task priorities for your analysis to hold (higher priority is assumed to be a larger number).

[6]

b) In the system as specified in a) the ~~3~~ 4 jobs are now known to share a common resource which requires mutually exclusive access from each job for 50 CPU cycles. Each job uses the resource once each job period, Thus now Job *B* spends 50 cycles accessing the resource, and 150 cycles doing other computation, in each job period. Assuming that priority inheritance protocol is used to control the mutually exclusive access: 11.00

(i) What is the maximum blocking number of cycles per period, as used in extended RMA analysis, for each job?

[4]

(ii) Recalculate the minimum safe CPU speed for the new system.

[2]

c) Consider again the system from part a), with no mutual exclusion. Assume that it runs on an RTOS with a separate task for each job. The CPU speed is now known to be 200 cycles/μs. The RTOS introduces an additional overhead of 100 cycles whenever tasks switch. This includes switching to/from the idle task. The RTOS also provides coroutines that can be defined within each task and switch with zero overhead.

(i) What is the maximum CPU utilisation of the task switch overhead, with one task for each job?

[2]

(ii) It is proposed that preemption threshold scheduling should be used in for this system. Describe how this scheduling method differs from priority scheduling. Determine, giving your reasons, the maximum preemption threshold for each job that will keep this system schedulable with each job terminating within its period. Explain how preemption threshold scheduling allows this system to run with fewer tasks than priority scheduling, and state the number of tasks required in this case. Estimate the CPU utilisation of the task switch overhead under high load conditions, stating any assumptions you make.

[6]

Job	CPU time / cycles	Period / μs
A	100	10
B	200	20
C	300	40
D	400	16

Figure 2.1

EE4.52
EE9 A0.5

Answer to Question 1

1.

a)

- Safety analysis unsound
  - software problems and safety task failures are not necessarily independent
  - typical use conditions do not stress system, or account for unusual user behaviour
  - simulation cannot be guaranteed to be identical to reality
  - hard deadlines mean that liveness problems such as priority inversion can cause failure but be dependent on small changes in operating conditions

[6]

- Design strategy inappropriate
  - Testing cannot in general find errors in real-time systems. Analysis is required.
  - Errors in field trials should be investigated and cured. Attempting to work round them is not acceptable in safety-critical applications because until the error is completely understood it may turn out to have new not yet discovered issues.

[4]

b) Recursive calling. Allows mutex protected subfunctions to be used safely inside code that protects using same mutex. Implementation requires number of mutex nested calls to be stored with mutex

Priority inheritance protocol. Solves priority inversion liveness problems. Requires dynamically changeable task priorities.

[could list other more complex protocols that solve priority inheritance]

[6]

c)

Assume CPU utilisation X2 (100% extra specified) and add (say) X1.5 (allowance for possible inter-task blocking). Note that number of tasks could increase as result of extra features. Therefore max CPU utilisation should be below RMA limit for infinite niumber of tasks, approx 70%. So total increase is:

X2X1.5X 80/70 = 3.4 times.

[4]

© Imperial College London

Answer to Question 2

a) A > D > B > C

utilisation:

A 10 cycles/us

B 10 cycles/us

C 7.5 cycles/us

D 25 cycles/us

total: 52.5 cycles/us

RMA limit for 4 tasks is 75.7% => CPU speed 69.4 cycles/us = 69,400,000 cycles/s.

[6]

b)

A,B,C: 50 cycles

D: 0 cycles.

[4]

Total now 52.5 + 50/10 + 50/20 + 50/40 = 61.25 cycles/us => 80,900,000 cycles/s

[2]

c)

(i) Each task gives max two new switches => 100/10+100/20+100/40+100/16 = 10+5+2.5+6.25 = 23.75 cycles/us = 11.875%.

[2]

(ii) PTS makes it more difficult (higher priority threshold) to preempt a started task than to start it. Each task has a preemption threshold (PT) >= its priority, and can be preempted only by tasks of higher priority then this threshold.

A > D > B > C. We determine highest feasible PT of each task starting with lowest priority (C).

At 200 cycles per second the time used in us is:

A: 0.5, B 1, C 1.5, D 2, switch: 0.5

Max delay introduced from C theshold is 2 us << 10us. Similarly all other tasks. Hence all PTs can be set to A priority, so that all jobs run to completion when started, and all jobs can be implemented as co-routines within one task. If this is done under high load conditions we have switch overhead of two switches per 40 us =>2.5%.

[6]

© Imperial College London